

SDK 开发



正点原子公司名称 : 广州市星翼电子科技有限公司

原子哥在线教学平台: www.yuanzige.com

开源电子网 / 论坛 : <http://www.openedv.com/forum.php>

正点原子淘宝店铺 : <https://zhengdianyuanzi.tmall.com>

正点原子官方网站 : www.alientek.com

正点原子 B 站视频 : <https://space.bilibili.com/394620890>

电话: 020-38271790 传真: 020-36773971

请关注正点原子公众号, 资料发布更新我们会通知。



扫码关注正点原子公众号

文档更新说明

版本	版本更新说明	负责人	校审	发布日期
V1.0	初稿:	邓志茂	正点原子	2025.03.22
V1.1	修改某些描述	邓志茂	正点原子	2025.05.28
V1.2	3.1 小节新增 RK3506J 开发板配置描述及 AMP、RGB 相关配置	邓志茂	正点原子	2025.06.27
V1.3	1. 删除 1.2 小节 dl 包安装 2. 修改 1.1 小节, 修改检出源码方法	邓志茂	正点原子	2025.07.18
V1.4	1. 修改第三章, 简化修改 defconfig 方法。	邓志茂	正点原子	2026.01.30
V1.5	1. 因更新 SDK, 文档也重新更新	邓志茂	正点原子	2026.03.05

目录

前言	5
第一章 SDK 源码安装及注意事项	7
1.1 SDK 源码安装	8
1.2 主机环境配置	8
1.3 特别注意事项	9
1.4 主机安装依赖库和工具	9
第二章 SDK 编译	10
2.1 SDK 目录介绍	11
2.2 build.sh 初探	11
2.3 选择 SDK 板级文件	14
2.4 SDK 全自动编译	17
2.5 生成的镜像文件	17
第三章 SDK 开发	20
3.1 SDK 板级配置文件	21
3.1.1 板级配置文件位置	21
3.1.2 板级配置文件详解	21
3.2 U-boot 开发	23
3.2.1 如何修改 defconfig	23
3.3 Kernel 开发	24
3.3.1 如何修改 defconfig	24
3.3.2 修改 dts	24
3.4 buildroot 开发	25
3.4.1 如何修改 defconfig	25
3.4.2 如何重新编译某个 Package	25
3.4.3 如何添加用户固件	26
3.5 如何烧写固件	26

前言

本文档介绍 ATK-DLRK3506 的 SDK 编译及使用。如果大家对内核/U-boot/Buildroot 等概念不了解的话请先去了解一下。此文档不再讲太多概念性的题话。也就意味着用户需要有一定的基础,这是必须的。因为 RK 的 SDK 参杂了非常多的东西,对于初学者一个 Buildroot 系统一个 Uboot 或者一个内核可能要讲很久才理解这是一个什么东西。可以参考正点原子阿尔法开发板教程,讲的很详细。

若确实用户基础很薄,笔者认为用户至少需要会对 Ubuntu 的一些操作指令熟练才能开发这个 SDK。比如解压,创建文件,修改文件等。

对于用户来说,无额外功能定制的情况下,不用开发内核 Uboot 及 buildroot,直接基于我们开发好的 SDK 直接做应用开发,可能会更省事。有额外功能添加的情况下,那么用户需要了解如何修改我们的 SDK,那么我们就来愉快开发 SDK 吧!

免责声明

本文档所提及的产品规格和使用说明仅供参考，如有内容更新，恕不另行通知；除非有特殊约定，本文档仅作为产品指导，所作陈述均不构成任何形式的担保。本文档版权归广州市星翼电子科技有限公司所有，未经公司的书面许可，任何单位和个人不得以营利为目的进行任何形式的传播。

为了得到最新版本的产品信息，请用户定时访问正点原子资料下载中心或者与淘宝正点原子旗舰店客服联系索取。感谢您的包容与支持。

第一章 SDK 源码安装及注意事项

本章介绍 SDK 源码安装及注意事项。

1.1 SDK 源码安装

安装 rk3506 Linux SDK, 开发板资料包中已经给用户提供了 SDK 的压缩包文件, 路径为: 开发板网盘 B 盘-虚拟机&SDK/02、ATK-DLRK3506 开发板 SDK/01、linux6.1_sdk/atk-dlrk3506_linux6.1_release_v1.x.x_xxxxxxxx.tar.gz 将其拷贝到家目录下(请注意, 初学者这里如果不会 Ubuntu 的相关拷贝文件操作请去看【开发板网盘 A 盘-基础资料\10、用户手册\02、辅助文档【正点原子】Ubuntu&Windows&Linux 开发板互传文件参考手册】)。

```
tar xf atk-dlrk3506_linux6.1_release_v1.3.0_20260304.tar.gz      # 解压需要几十秒
ls -a
```

```
allientek@allientek:~$ tar xf atk-dlrk3506_linux6.1_release_v1.3.0_20260304.tar.gz
allientek@allientek:~$ cd atk-dlrk3506_linux6.1_release_v1.3.0_20260304
allientek@allientek:~/atk-dlrk3506_linux6.1_release_v1.3.0_20260304$ ls -a
.  ..  使用说明.txt  buildroot  .repo  repo.sh
```

隐藏文件

下面我们来检出源码, 至于为什么是一个.repo 隐藏文件, 因为 RK (多 git 仓库) 就是用 repo 来管理多个 git 仓库的。

执行 ./repo.sh 检出源码,

```
./repo.sh      #检出脚本
```

./repo.sh 检出脚本内容如下, 用户也可以直接拷贝下面指令来检出 SDK 源码。

```
repo/repo/repo sync -l -j2      #这里的-l 为-L, L 的小写字母 l
```

```
allientek@allientek:~/atk-dlrk3506_linux6.1_release_v1.3.0_20260304$ ./repo.sh
正在更新文件: 100% (266/266), 完成.
正在更新文件: 100% (20051/20051), 完成.
正在更新文件: 100% (260/260), 完成.
正在更新文件: 100% (1144/1144), 完成.
正在更新文件: 100% (13926/13926), 完成.
正在更新文件: 100% (41/41), 完成.
正在更新文件: 100% (366/366), 完成.o/poky正在更新文件: 34% (125/366)
正在更新文件: 100% (206/206), 完成.
正在更新文件: 100% (5790/5790), 完成.
正在更新文件: 100% (33/33), 完成.
正在更新文件: 100% (83812/83812), 完成.
正在更新文件: 100% (5722/5722), 完成.
正在更新文件: 100% (6910/6910), 完成.
正在更新文件: 100% (41306/41306), 完成.
Checking out: 100% (40/40), done in 1m47.369s
repo sync has finished successfully.
allientek@allientek:~/atk-dlrk3506_linux6.1_release_v1.3.0_20260304$
```

1.2 主机环境配置

使用指定的 Ubuntu20.04 版本, 高版本的 Ubuntu 如 Ubuntu22 可能编译不通过, 我们只在 Ubuntu20.04 测试。当然如果用户对 Ubuntu 非常熟悉, 可以尝试用其他版本 Ubuntu 编译。初学者一般都是用指定版本即可。Ubuntu 系统可用磁盘容量至少 100G。当然越可用的容量越多越好。

虚拟机分配内存至少 8G, 越多越好, 核心数, 能分多就多分! 实在不行的话, 4GB 内存也可以试试, 同时你可以设置 swap 交换空间, 参考网盘资料下用户手册/辅助文档下的《【正点原子】Ubuntu 扩充 swap 交换空间》手册配置交换空间试试!

至于编译时间, 不报错的情况下通常 1.5~2 个小时。与磁盘访问速度及 CPU 性能有关。

1.3 特别注意事项

1. 不要用 root 用户编译/解压 SDK 源码!
2. 不要用 sudo 权限来解压 SDK 源码!
3. 源码也不能解压至包含中文字符的路径下!

1.4 主机安装依赖库和工具

```
sudo apt-get update # 先更新软件源
sudo apt-get install git ssh make gcc libssl-dev \
liblz4-tool expect expect-dev g++ patchelf chrpath gawk texinfo chrpath \
diffstat binfmt-support qemu-user-static live-build bison flex fakeroot \
cmake gcc-multilib g++-multilib unzip device-tree-compiler ncurses-dev \
bzip2 expat gpgv2 cpp-aarch64-linux-gnu libgmp-dev \
libmpc-dev bc python-is-python3 python2
```

注意若使用其他版本 Ubuntu 请根据安装包名称采用对应的安装命令,若编译遇到报错,可以视报错信息,安装对应的软件包。

```
alientek@alientek:~/atk-dlrk3506_linux6.1_release_v1.3.0_20260304$ sudo apt-get update
获取:1 http://security.ubuntu.com/ubuntu focal-security InRelease [128 kB]
获取:2 http://security.ubuntu.com/ubuntu focal-security/main amd64 DEP-11 Metadata [74.9 kB]
获取:3 http://security.ubuntu.com/ubuntu focal-security/multiverse amd64 DEP-11 Metadata [940 B]
获取:4 http://security.ubuntu.com/ubuntu focal-security/restricted amd64 DEP-11 Metadata [212 B]
获取:5 http://security.ubuntu.com/ubuntu focal-security/universe amd64 DEP-11 Metadata [160 kB]
获取:6 https://packages.microsoft.com/repos/code stable InRelease [3,590 B]
获取:7 https://packages.microsoft.com/repos/code stable/main armhf Packages [23.5 kB]
获取:8 https://packages.microsoft.com/repos/code stable/main arm64 Packages [23.4 kB]
获取:9 https://packages.microsoft.com/repos/code stable/main amd64 Packages [23.4 kB]
命中:10 http://mirrors.aliyun.com/ubuntu focal InRelease
获取:11 http://mirrors.aliyun.com/ubuntu focal-security InRelease [128 kB]
获取:12 http://mirrors.aliyun.com/ubuntu focal-security/main amd64 DEP-11 Metadata [74.6 kB]
获取:13 http://mirrors.aliyun.com/ubuntu focal-security/restricted amd64 DEP-11 Metadata [212 B]
获取:14 http://mirrors.aliyun.com/ubuntu focal-security/universe amd64 DEP-11 Metadata [160 kB]
获取:15 http://mirrors.aliyun.com/ubuntu focal-security/multiverse amd64 DEP-11 Metadata [940 B]
已下载 491 kB, 耗时 30秒 (16.2 kB/s)
正在读取软件包列表... 完成
alientek@alientek:~/atk-dlrk3506_linux6.1_release_v1.3.0_20260304$ sudo apt-get install git ssh make gcc libssl-dev \
> liblz4-tool expect expect-dev g++ patchelf chrpath gawk texinfo chrpath \
> diffstat binfmt-support qemu-user-static live-build bison flex fakeroot \
> cmake gcc-multilib g++-multilib unzip device-tree-compiler ncurses-dev \
> bzip2 expat gpgv2 cpp-aarch64-linux-gnu libgmp-dev \
> libmpc-dev bc python-is-python3 python2
正在读取软件包列表... 完成
正在分析软件包的依赖关系树
正在读取状态信息... 完成
注意, 选中 'expect' 而非 'expect-dev'
注意, 选中 'libncurses-dev' 而非 'ncurses-dev'
bc 已经是最新版 (1.07.1-2build1)。
bc 已设置为手动安装。
bison 已经是最新版 (2:3.5.1+dfsg-1)。
bzip2 已经是最新版 (1.0.8-2)。
bzip2 已设置为手动安装。
cmake 已经是最新版 (3.16.3-1ubuntu1)。
device-tree-compiler 已经是最新版 (1.5.1-1)。
```

第二章 SDK 编译

本章编译 SDK。

2.1 SDK 目录介绍

SDK 源码根目录下包含有 app、buildroot、device、external、tools、u-boot 等多个目录，每个目录或其子目录会对应一个 git 工程；因为 SDK 的代码和相关文档被划分成了若干 git 仓库分别进行版本管理（所以 SDK 实际上包含有若干 git 仓库），它们按照功能、所属模块划分，分别组织到不同的目录下。

- app: 存放上层应用 app，以及其它的 C/C++ 应用程序。
- buildroot: 基于 buildroot 开发的根文件系统。
- device/rockchip: 存放各芯片板级配置文件和 Parameter 分区表文件，以及一些编译与打包固件的脚本和预备文件。
- docs: 存放芯片模块开发指导文档、平台支持列表、芯片平台相关文档、Linux 开发指南等。
- external: 存放所需的第三方库，包括音频、视频、网络、recovery 等。
- kernel: Linux 6.1 版本内核源码。
- prebuilts: 存放交叉编译工具链。
- rkbin: 存放 Rockchip 相关的 Binary 和工具。
- rockdev: 存放编译输出固件，编译 SDK 后才会生成该文件夹。
- tools: 存放 Linux 和 Windows 操作系统环境下常用的工具，包括镜像烧录工具、SD 卡升级启动制作工具、批量烧录工具等，譬如前面给大家介绍的 RKDevTool 工具以及 Linux_Upgrade_Tool 工具都存放在该目录。
- u-boot: 基于 v2017.09 版本进行开发的 uboot 源码。
- yocto: 基于 Yocto 开发的根文件系统。国内用 Buildroot 较多，对 Yocto 感兴趣的可自行研究。

注：还有一些是链接文件，这里就不写出来了。

```
alientek@alientek:~/atk-dlrk3506_linux6.1_release_v1.3.0_20260304$ ls
使用说明.txt app buildroot build.sh device docs external hal kernel kernel-6.1 Makefile prebuilts repo.sh rkbin rkflash.sh rtos tools u-boot yocto
alientek@alientek:~/atk-dlrk3506_linux6.1_release_v1.3.0_20260304$
```

2.2 build.sh 初探

build.sh 是 RK 提供的一个编译脚本，使用该脚本可以方便用户快速构建出各种镜像文件以及对镜像进行打包操作，既可以一键全自动编译整个 SDK，也可以单独编译 U-Boot、Linux Kernel、buildroot 等，非常方便！

build.sh 脚本其实是一个软链接文件，实际指向了 device/rockchip/common/build.sh 文件。可通过执行如下命令查看 build.sh 脚本的使用方法：

```
./build.sh -h
```

用法如下，比如用户需要清除内核，那么执行 `./build.sh clean-kernel` 即可。更多用法，用到时用户自行学习即可。

```
##### Rockchip Linux SDK #####

Manifest: atk-dlrk3506_linux6.1_release_v1.3.0_20260304.xml

Log colors: message notice warning error fatal

Usage: build.sh [OPTIONS]
```

Available options:

chip[:<chip>[:<config>]]

choose chip

defconfig[:<config>]

choose defconfig

*_defconfig

switch to specified defconfig

available defconfigs:

01_atk_dlrk3506b_mipi720x1280_nand_ubi_squashfs_defconfig
02_atk_dlrk3506b_mipi720x1280_nand_ubi_squashfs_thunderboot_defconfig
03_atk_dlrk3506b_mipi720x1280_nand_ubi_ubifs_amp_linux_defconfig
04_atk_dlrk3506b_mipi720x1280_nand_ubi_ubifs_amp_mcu_defconfig
05_atk_dlrk3506b_mipi720x1280_nand_ubi_ubifs_defconfig
06_atk_dlrk3506b_rgb1024x600_nand_ubi_ubifs_defconfig
07_atk_dlrk3506b_mipi720x1280_nand_ubi_ubifs_thunderboot_defconfig
08_atk_dlrk3506b_mipi720x1280_sd_defconfig
09_atk_dlrk3506j_mipi720x1280_emmc_amp_linux_defconfig
10_atk_dlrk3506j_mipi720x1280_emmc_amp_mcu_defconfig
11_atk_dlrk3506j_mipi720x1280_emmc_defconfig
12_atk_dlrk3506j_mipi720x1280_emmc_thunderboot_defconfig
13_atk_dlrk3506j_mipi720x1280_nand_ubi_squashfs_defconfig
14_atk_dlrk3506j_mipi720x1280_nand_ubi_squashfs_thunderboot_defconfig
15_atk_dlrk3506j_mipi720x1280_nand_ubi_ubifs_amp_linux_defconfig
16_atk_dlrk3506j_mipi720x1280_nand_ubi_ubifs_amp_mcu_defconfig
17_atk_dlrk3506j_mipi720x1280_nand_ubi_ubifs_defconfig
18_atk_dlrk3506j_mipi720x1280_nand_ubi_ubifs_thunderboot_defconfig
19_atk_dlrk3506b_automipi_nand_ubi_squashfs_defconfig
20_atk_dlrk3506b_automipi_sd_defconfig
21_atk_dlrk3506j_automipi_nand_ubi_squashfs_defconfig
22_atk_dlrk3506j_automipi_emmc_defconfig
23_atk_dlrk3506b_mipi800x1280_nand_ubi_squashfs_defconfig
24_atk_dlrk3506b_mipi800x1280_nand_ubi_squashfs_thunderboot_defconfig
25_atk_dlrk3506b_mipi800x1280_nand_ubi_ubifs_amp_linux_defconfig
26_atk_dlrk3506b_mipi800x1280_nand_ubi_ubifs_amp_mcu_defconfig
27_atk_dlrk3506b_mipi800x1280_nand_ubi_ubifs_defconfig
28_atk_dlrk3506b_mipi800x1280_sd_defconfig
29_atk_dlrk3506j_mipi800x1280_emmc_amp_linux_defconfig
30_atk_dlrk3506j_mipi800x1280_emmc_amp_mcu_defconfig
31_atk_dlrk3506j_mipi800x1280_emmc_defconfig
32_atk_dlrk3506j_mipi800x1280_emmc_thunderboot_defconfig
33_atk_dlrk3506j_mipi800x1280_nand_ubi_squashfs_defconfig
34_atk_dlrk3506j_mipi800x1280_nand_ubi_squashfs_thunderboot_defconfig
35_atk_dlrk3506j_mipi800x1280_nand_ubi_ubifs_amp_linux_defconfig
36_atk_dlrk3506j_mipi800x1280_nand_ubi_ubifs_amp_mcu_defconfig
37_atk_dlrk3506j_mipi800x1280_nand_ubi_ubifs_defconfig
38_atk_dlrk3506j_mipi800x1280_nand_ubi_ubifs_thunderboot_defconfig

39_atk_dlrk3506j_rgb1024x600_emmc_defconfig	
40_atk_dlrk3506j_rgb1024x600_nand_ubi_ubifs_defconfig	
41_atk_dlrk3506b_automipi_nand_ubi_ubifs_defconfig	
42_atk_dlrk3506j_automipi_nand_ubi_ubifs_defconfig	
olddefconfig	resolve any unresolved symbols in .config
savedefconfig	save current config to defconfig
menuconfig	interactive curses-based configurator
config	modify SDK defconfig
print-parts	print partitions
list-parts	alias of print-parts
mod-parts	interactive partition table modify
edit-parts	edit raw partitions
new-parts:<offset>:<name>:<size>...	re-create partitions
insert-part:<idx>:<name>[:<size>]	insert partition
del-part:(<idx> <name>)	delete partition
move-part:(<idx> <name>):<idx>	move partition
rename-part:(<idx> <name>):<name>	rename partition
resize-part:(<idx> <name>):<size>	resize partition
misc	pack misc image
kernel-6.1[:dry-run]	build kernel 6.1
kernel[:dry-run]	build kernel
recovery-kernel[:dry-run]	build kernel for recovery
kernel-modules[:<dst dir>:dry-run]	build kernel modules
modules[:<dst dir>:dry-run]	alias of kernel-modules
linux-headers[:<arch>:dry-run]	build linux-headers
kernel-config[:dry-run]	modify kernel defconfig
kconfig[:dry-run]	alias of kernel-config
kernel-make[:<arg1>:<arg2>]	run kernel make
kmake[:<arg1>:<arg2>]	alias of kernel-make
wifibt[:<dst dir>[:<chip>]]	build Wifi/BT
amp	build and pack amp system
buildroot-config[:<config>]	modify buildroot defconfig
bconfig[:<config>]	alias of buildroot-config
buildroot-make[:<arg1>:<arg2>]	run buildroot make
bmake[:<arg1>:<arg2>]	alias of buildroot-make
buildroot-sdk	build the buildroot SDK tarball
bsdk	alias of buildroot-sdk
rootfs[:<rootfs type>]	build default rootfs
buildroot	build buildroot rootfs
yocto	build yocto rootfs
debian	build debian rootfs
recovery	build recovery
security-createkeys	create keys for security

security-misc	build misc with system encryption key
security-ramboot[:system_image]	build security ramboot
security-system[:system_image]	build security system
loader[:dry-run]	build loader (u-boot)
uboot[:dry-run]	build u-boot
u-boot[:dry-run]	alias of uboot
extra-parts	pack extra partition images
firmware	pack and check firmwares
edit-package-file	edit package-file
edit-ota-package-file	edit package-file for OTA
updateimg	build update image
ota-updateimg	build update image for OTA
all	build images
release[:<subdir>[:<name>]]	release images and build info
all-release[:<subdir>[:<name>]]	build and release images
shell	setup a shell for developing
buildroot-shell	setup a shell for buildroot developing
bshell	alias of buildroot-shell
yocto-shell	setup a shell for yocto developing
yshell	alias of yocto-shell
cleanall	cleanup all
clean-misc	cleanup misc
clean-firmware	cleanup firmware
clean-recovery	cleanup recovery
clean-rootfs	cleanup rootfs
clean-kernel	cleanup kernel
clean-config	cleanup config
clean-security	cleanup security
clean-extra-parts	cleanup extra-parts
clean-updateimg	cleanup updateimg
clean-loader	cleanup loader
clean-amp	cleanup amp
post-rootfs <rootfs dir>	trigger post-rootfs hook scripts
help	display this information

Default option is 'all'.

2.3 选择 SDK 板级文件

本小节向用户介绍如何选择 atk-dlrk3506 Linux SDK, 首先进入到 SDK 源码根目录下, 在编译之前先执行如下命令指定 SDK 的板级配置文件:

进入 SDK 源码顶层目录, 执行下面的指令, 选择板级配置文件。(初学者会问为什么要做这一步? 答: 因为我们要告诉 SDK, 我们要编译何种类型的镜像。同时多个配置代表不同功能, 需要用户慢慢了解。)

./build.sh lunch

执行了上面指令会要求你输入一个序号来选择板级配置文件。这种板级文件的作用是配置不同的设备树、不同的内核配置和不同的文件系统类型以适配不同类型的开发板。

```

1. 01_atk_dlrk3506b_mipi720x1280_nand_ubi_squashfs_defconfig
2. 02_atk_dlrk3506b_mipi720x1280_nand_ubi_squashfs_thunderboot_defconfig
3. 03_atk_dlrk3506b_mipi720x1280_nand_ubi_ubifs_amp_linux_defconfig
4. 04_atk_dlrk3506b_mipi720x1280_nand_ubi_ubifs_amp_mcu_defconfig
5. 05_atk_dlrk3506b_mipi720x1280_nand_ubi_ubifs_defconfig
6. 06_atk_dlrk3506b_rgb1024x600_nand_ubi_ubifs_defconfig
7. 07_atk_dlrk3506b_mipi720x1280_nand_ubi_ubifs_thunderboot_defconfig
8. 08_atk_dlrk3506b_mipi720x1280_sd_defconfig
9. 09_atk_dlrk3506j_mipi720x1280_emmc_amp_linux_defconfig
10. 10_atk_dlrk3506j_mipi720x1280_emmc_amp_mcu_defconfig
11. 11_atk_dlrk3506j_mipi720x1280_emmc_defconfig
12. 12_atk_dlrk3506j_mipi720x1280_emmc_thunderboot_defconfig
13. 13_atk_dlrk3506j_mipi720x1280_nand_ubi_squashfs_defconfig
14. 14_atk_dlrk3506j_mipi720x1280_nand_ubi_squashfs_thunderboot_defconfig
15. 15_atk_dlrk3506j_mipi720x1280_nand_ubi_ubifs_amp_linux_defconfig
16. 16_atk_dlrk3506j_mipi720x1280_nand_ubi_ubifs_amp_mcu_defconfig
17. 17_atk_dlrk3506j_mipi720x1280_nand_ubi_ubifs_defconfig
18. 18_atk_dlrk3506j_mipi720x1280_nand_ubi_ubifs_thunderboot_defconfig
19. 19_atk_dlrk3506b_autonipi_nand_ubi_squashfs_defconfig
20. 20_atk_dlrk3506b_autonipi_sd_defconfig
21. 21_atk_dlrk3506j_autonipi_nand_ubi_squashfs_defconfig
22. 22_atk_dlrk3506j_autonipi_emmc_defconfig
23. 23_atk_dlrk3506b_mipi800x1280_nand_ubi_squashfs_defconfig
24. 24_atk_dlrk3506b_mipi800x1280_nand_ubi_squashfs_thunderboot_defconfig
25. 25_atk_dlrk3506b_mipi800x1280_nand_ubi_ubifs_amp_linux_defconfig
26. 26_atk_dlrk3506b_mipi800x1280_nand_ubi_ubifs_amp_mcu_defconfig
27. 27_atk_dlrk3506b_mipi800x1280_nand_ubi_ubifs_defconfig
28. 28_atk_dlrk3506b_mipi800x1280_sd_defconfig
29. 29_atk_dlrk3506j_mipi800x1280_emmc_amp_linux_defconfig
30. 30_atk_dlrk3506j_mipi800x1280_emmc_amp_mcu_defconfig
31. 31_atk_dlrk3506j_mipi800x1280_emmc_defconfig
32. 32_atk_dlrk3506j_mipi800x1280_emmc_thunderboot_defconfig
33. 33_atk_dlrk3506j_mipi800x1280_nand_ubi_squashfs_defconfig
34. 34_atk_dlrk3506j_mipi800x1280_nand_ubi_squashfs_thunderboot_defconfig
35. 35_atk_dlrk3506j_mipi800x1280_nand_ubi_ubifs_amp_linux_defconfig
36. 36_atk_dlrk3506j_mipi800x1280_nand_ubi_ubifs_amp_mcu_defconfig
37. 37_atk_dlrk3506j_mipi800x1280_nand_ubi_ubifs_defconfig
38. 38_atk_dlrk3506j_mipi800x1280_nand_ubi_ubifs_thunderboot_defconfig
39. 39_atk_dlrk3506j_rgb1024x600_emmc_defconfig
40. 40_atk_dlrk3506j_rgb1024x600_nand_ubi_ubifs_defconfig
41. 41_atk_dlrk3506b_autonipi_nand_ubi_ubifs_defconfig
42. 42_atk_dlrk3506j_autonipi_nand_ubi_ubifs_defconfig
Which would you like? [1]: 19

```

atk_dlrk3506b	正点原子 atk_dlrk3506b 开发板, b 为商业级
atk_dlrk3506j	正点原子 atk_dlrk3506j 开发板, j 为工业级
autonipi	自动识别 MIPI 屏幕, 通过 HWID, 仅识别正点原子 5.5 寸 720x1280 和 10.1 寸 800x1280 分辨率 MIPI 屏幕
mipi720x1280	正点原子 5.5 寸 720x1280 分辨率 MIPI 屏幕
mipi800x1280	正点原子 10.1 寸 800x1280 分辨率 MIPI 屏幕
rgb1024x600	驱动正点原子 7 寸 RGB 屏幕 1024x600 的配置。RK3506B 需要搭配转接板测试, 转接板原理图位于网盘 PCB 位于 07、硬件资料08、ATK-DLRK3506B RGB888 方案转接板。RK3506J 则需要自己画底板, 没有转接板。
nand	存储为 SPI NANDFLASH, 从 SPI NANDFLASH 启动
emmc	存储为 eMMC, 从 eMMC 启动
sd	存储为 SD 卡 (即 TF 卡), 从 SD 卡启动
ubi	ubi 类型文件系统
ubi_squashfs	squashfs 类型文件系统, 只读文件系统类型, 生成的镜像压缩率高。此种镜像更有效利用存储空间, 并且根文件系统只读, 禁止频繁读写 rootfs, 降低因读写导致分区损坏, 对于 NandFlash 一般生产发布用此种类型文件系统

ubi_ubifs	可读写文件系统，主要针对 NAND 区别。SD 卡和 EMMC 默认都为读写文件系统
thunderboot	快速启动配置，注意由于出厂的 UI 启动位置排在较后，所以在启动时并没有感到明显时间变化，此配置是跳过 Uboot，使用 SPL 来引导内核。请参考 LVGL 开发手册，快速启动 RK 默认是针对 LVGL 的
amp_linux	非对称多处理系统，此配置两个 A7 核心 Linux (smp)，一个跑 RTT 实时操作系统。
amp_mcu	非对称多处理系统，此配置三个 A7 核心 Linux (smp)，M0 跑 Bare-metal 裸机。
defconfig	后缀，配置文件

实际上执行 build.sh luch 执行时也会提示。

```

allientek@allientek:~/atk-dlrk3506_linux6.1_release_v1.3.0_20260304$ ./build.sh lunch
##### Rockchip Linux SDK #####
Manifest: atk-dlrk3506_linux6.1_release_v1.3.0_20260304.xml
Log colors: message notice warning error fatal
===== 正点原子RK3506 快速配置选择 =====

配置分类说明:
atk_dlrk3506b: 正点原子RK3506B开发板
atk_dlrk3506j: 正点原子RK3506J开发板
mipi720x1280: 720x1280分辨率MIPI屏幕
mipi800x1280: 800x1280分辨率MIPI屏幕
rgb1024x600: 1024x600分辨率RGB屏幕
amp_linux: AMP Linux系统镜像 (运行在A核)
amp_mcu: AMP MCU镜像 (运行在M核)
thunderboot: 快速启动版
nand: NAND闪存
emmc: EMMC存储
sd: SD卡存储
automipi: 自动识别正点原子MIPI屏
ubi_ubifs: 可读写文件系统, 调试使用
ubi_squashfs: 只读压缩文件系统, 用于生产

使用说明:
1. 有/无屏用户: 根据自己的开发板选择, 新手推荐19、21、22、41、42中的automipi配置
2. 有正点原子MIPI屏: 根据屏幕分辨率选择对应配置
3. RGB屏配置: 底板上默认无RGB接口, 需要自己做底板
4. 自动适配屏幕: 选择automipi配置(仅识别正点原子5.5寸720p和10.1寸800p屏幕)
5. NAND存储读写文件系统: 选择带ubi_ubifs(读写)的配置, 常用于临时调试
6. NAND存储只读文件系统: 选择带squashfs(只读)的配置, 常用于生产
7. 开发AMP多核异构:
   - amp_linux: AMP Linux系统镜像 (运行在A核)
   - amp_mcu: AMP MCU镜像 (运行在M核)
8. 快速启动需求: 选择带thunderboot的配置
9. SD卡存储: 选择带SD卡的配置
10. SDK更多开发资料参考网盘资料10、用户手册/【正点原子】ATK-DLRK3506_SDK开发手册

请输入配置序号 (1-42):

```

请根据用户的开发板类型选择配置文件，请区分 RK3506J 和 RK3506B，这两个为不同类型开发板，外设不一样。同时注意核心板上的存储器类型，是 NandFlash 就选择带“nand”字段的配置，是 eMMC 的就带“emmc ”配置。

例 RK3506B 存储为 NAND 的核心板，需要选择自动识别正点原子 MIPI 屏幕、只读根文件系统。对应的就是 19_atk_dlrk3506b_automipi_nand_ubi_squashfs_defconfig 对应的数字为 19，那么输入 19 即可。**关于屏幕不管用户有没有屏幕都是可以正常启动的。**

- RK3506B NAND 出厂系统选择的就是 19（只读文件系统）
- RK3506J EMMC 版本出厂系统选择的是 22。
- RK3506J NAND 出厂系统选择的是 21（只读文件系统）。

用户可以随意选择 1~42 编译，到测试时可以再执行 ./build.sh 选择切换其他配置，不用重新清除编译，非常方便！

2.4 SDK 全自动编译

在上一步选择了板级配置后（必须，需要执行一次，下次不用再选择，可直接./build.sh），我们直接执行下面指令开始编译 SDK。直接执行下面指令编译，编译全部。

./build.sh

```
33. 33_atk_dlrk3506j_mipi800x1280_nand_ubi_squashfs_defconfig
34. 34_atk_dlrk3506j_mipi800x1280_nand_ubi_squashfs_thunderboot_defconfig
35. 35_atk_dlrk3506j_mipi800x1280_nand_ubi_ubifs_omp_linux_defconfig
36. 36_atk_dlrk3506j_mipi800x1280_nand_ubi_ubifs_omp_ncu_defconfig
37. 37_atk_dlrk3506j_mipi800x1280_nand_ubi_ubifs_defconfig
38. 38_atk_dlrk3506j_mipi800x1280_nand_ubi_ubifs_thunderboot_defconfig
39. 39_atk_dlrk3506j_rgb1024x600_ewmc_defconfig
40. 40_atk_dlrk3506j_rgb1024x600_nand_ubi_ubifs_defconfig
41. 41_atk_dlrk3506b_automipi_nand_ubi_ubifs_defconfig
42. 42_atk_dlrk3506j_automipi_nand_ubi_ubifs_defconfig
Which would you like? [1]: 19
Switching to defconfig: /home/alientek/atk-dlrk3506_linux6.1_release_v1.3.0_20260304/device/rockchip/.chip/19_atk_dlrk3506b_automipi_nand_ubi_squashfs_defconfig
#
# configuration written to /home/alientek/atk-dlrk3506_linux6.1_release_v1.3.0_20260304/output/.config
#
Using last kernel version(6.1)
alientek@alientek:~/atk-dlrk3506_linux6.1_release_v1.3.0_20260304$ ./build.sh
##### Rockchip Linux SDK #####
Manifest: atk-dlrk3506_linux6.1_release_v1.3.0_20260304.xml
Log colors: message notice warning error fatal
Log saved at /home/alientek/atk-dlrk3506_linux6.1_release_v1.3.0_20260304/output/sessions/2026-03-04_17-53-53
Using last kernel version(6.1)
=====
Final configs
=====
RK_BOOT_COMPRESSED=y
RK_BOOT_FIT_ITS=/home/alientek/atk-dlrk3506_linux6.1_release_v1.3.0_20260304/device/rockchip/.chip/zboot.its
RK_BOOT_TWC=zboot.its
```

经过大约 1 个半小时左右，编译完成，如下图。

```
boot boot.img
rootfs rootfs.img
userdata userdata.img
Packing /home/alientek/atk-dlrk3506_linux6.1_release_v1.3.0_20260304/output/firmware/update.img for update...
Android Firmware Package Tool v2.29
----- PACKAGE -----
Add file: ./package-file
package-file,Add file: ./package-file done,offset=0x800,size=0xc5,userspace=0x1
Add file: ./parameter.txt
parameter,Add file: ./parameter.txt done,offset=0x1000,size=0x1d1,userspace=0x1,flash_address=0x00000000
Add file: ./MiniLoaderAll.bin
bootloader,Add file: ./MiniLoaderAll.bin done,offset=0x1800,size=0x421c0,userspace=0x85
Add file: ./uboot.img
uboot,Add file: ./uboot.img done,offset=0x44000,size=0x40000,userspace=0x800,flash_address=0x00002000
Add file: ./misc.img
misc,Add file: ./misc.img done,offset=0x444000,size=0xc000,userspace=0x18,flash_address=0x00004000
Add file: ./recovery.img
recovery,Add file: ./recovery.img done,offset=0x450000,size=0xdb3400,userspace=0x1b67,flash_address=0x00004a00
Add file: ./boot.img
boot,Add file: ./boot.img done,offset=0x1203800,size=0x5e4a00,userspace=0xbca,flash_address=0x0000ba00
Add file: ./rootfs.img
rootfs,Add file: ./rootfs.img done,offset=0x17e8800,size=0x6580000,userspace=0xc00,flash_address=0x00010a00
Add file: ./userdata.img
userdata,Add file: ./userdata.img done,offset=0x7d68800,size=0xc80000,userspace=0x1900,flash_address=0x0006aa00
Add CRC...
Make firmware OK!
----- OK -----
*****rkImageMaker ver 2.29*****
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Run 'make edit-package-file' if you want to change the package-file.
Running mk-updateimg.sh - build_updateimg succeeded.
Images under /home/alientek/atk-dlrk3506_linux6.1_release_v1.3.0_20260304/output/firmware/ are ready!
Running mk-firmware.sh - build_firmware succeeded.
Running 99-all.sh - build_all succeeded.
alientek@alientek:~/atk-dlrk3506_linux6.1_release_v1.3.0_20260304$
```

2.5 生成的镜像文件

生成的镜像文件在 SDK/rockdev 文件夹。由于 rockdev 这个是一个软链接文件，它指向的是 SDK/output/firmware/这个文件夹，如下图。

```
alientek@alientek:~/atk-dlrk3506_linux6.1_release_v1.3.0_20260304$ ls output/firmware/
boot.img MiniLoaderAll.bin misc.img oem.img parameter.txt recovery.img rootfs.img uboot.img update.img userdata.img
alientek@alientek:~/atk-dlrk3506_linux6.1_release_v1.3.0_20260304$
```

镜像文件介绍，如下表所示：

镜像名称	作用
uboot.img	uboot.img 是一种 FIT 格式镜像，它由多个镜像合并而成，其中包括 trust 镜像（ARM Trusted Firmware + OP-TEE OS）、u-boot 镜像、u-boot dtb；编译 U-Boot 时会将这些镜像打包成一个 uboot.img。 uboot.img 会烧录到开发板 uboot 分区
boot.img	boot.img 也是一种 FIT 格式镜像，它也是由多个镜像合并而成，其中包括内核镜像、内核 DTB、资源镜像 resource.img。 boot.img 会烧录到开发板 boot 分区
MiniLoaderAll.bin	该镜像是在运行在 RK3506 平台 U-Boot 之前的一段 Loader 代码（也就是比 U-Boot 更早阶段的 Loader），MiniLoaderAll.bin 由 TPL 和 SPL 两部分组成，TPL 用于初始化 DDR，运行在 SRAM；而 SPL 运行在 DDR，主要负责加载、引导 uboot.img。
misc.img	包含 BCB（Bootloader Control Block）信息，该镜像会烧写到开发板 misc 分区。 misc 分区是一个很重要的分区，其中存放了 BCB 数据块，主要用于 Android/Linux 系统、U-Boot 以及 recovery 之间的通信
oem.img	给厂家使用，用于存放厂家的 APP 或数据，该镜像会烧写至开发板 oem 分区，系统启动之后会将其挂载到/oem 目录。注意由于 RK3506 NandFlash 大小限制，虽然编译出了 oem.img，但是不烧写 oem.img，也就是不会有/oem 挂载目录。所有 App 或者用户数据建议存放/userdata 分区下！
parameter.txt	一个 txt 文本文件，是 RK3506 平台的分区表文件（记录分区名以及每个分区它的起始地址、结束地址）；烧写镜像时，并不需要将 parameter.txt 文件烧写到 Flash，而是会读取它的信息去定义分区。
recovery.img	recovery 模式镜像，recovery.img 用于进入 recovery 模式，recovery.img 会烧录到开发板 recovery 分区。 recovery 模式是一种用于对设备进行修复、升级更新的模式。recovery.img 也是 FIT 格式镜像，也是由多个镜像合并而成，其中包括 ramdisk（进入 recovery 模式时挂载该根文件系统）、内核镜像（进入 recovery 模式时启动该内核镜像）、内核 DTB 以及 resource.img。
rootfs.img	正常启动模式下对应的根文件系统镜像，包含大量的库文件、可执行文件等。 rootfs.img 会烧录到开发板 rootfs 分区
userdata.img	给用户使用，可用于存放用户的 App 或数据；该镜像会烧写至开发板 userdata 分区，系统启动之后，会将其挂载到/userdata 目录
update.img	uboot.img、boot.img、MiniLoaderAll.bin、parameter.txt、misc.img、rootfs.img、oem.img、userdata.img、recovery.img 等打包成一个 update.img 固件。

好了现在我们已经编译完成 SDK 了, 可以使用 RKDevTool 参考网盘资料下用户手册/辅助文档下的出厂镜像烧录文档烧录即可。

第三章 SDK 开发

本章带大家快速开发 SDK。

3.1 SDK 板级配置文件

3.1.1 板级配置文件位置

在 2.3 小节中，我们选择了板级配置文件。这些板级配置文件位于 `<SDK>/device/rockchip/rk3506/` 下，如下图所示。

```
alientek@alientek:~/atk-dlrk3506_linux64_release-v1.3.0_20200304/device/rockchip/rk3506$ ls
01_atk_dlrk3506b_mipi720x1280_nand_ubi_squashfs_defconfig  21_atk_dlrk3506j_autonip1_nand_ubi_squashfs_defconfig  41_atk_dlrk3506b_autonip1_nand_ubi_ubifs_defconfig
02_atk_dlrk3506b_mipi720x1280_nand_ubi_squashfs_thunderboot_defconfig  22_atk_dlrk3506j_autonip1_emmc_defconfig  42_atk_dlrk3506j_autonip1_nand_ubi_ubifs_defconfig
03_atk_dlrk3506b_mipi720x1280_nand_ubi_ubifs_amlinux_defconfig  23_atk_dlrk3506b_mipi800x1280_nand_ubi_squashfs_defconfig  43_atk_dlrk3506j_autonip1_nand_ubi_ubifs_defconfig
04_atk_dlrk3506b_mipi720x1280_nand_ubi_ubifs_amlinux_defconfig  24_atk_dlrk3506b_mipi800x1280_nand_ubi_squashfs_thunderboot_defconfig  44_atk_dlrk3506j_autonip1_nand_ubi_ubifs_defconfig
05_atk_dlrk3506b_mipi720x1280_nand_ubi_ubifs_amlinux_defconfig  25_atk_dlrk3506b_mipi800x1280_nand_ubi_ubifs_amlinux_defconfig  45_atk_dlrk3506j_autonip1_nand_ubi_ubifs_defconfig
06_atk_dlrk3506b_rpi024x600_nand_ubi_ubifs_defconfig  26_atk_dlrk3506b_mipi800x1280_nand_ubi_ubifs_amlinux_defconfig  46_atk_dlrk3506j_autonip1_nand_ubi_ubifs_defconfig
07_atk_dlrk3506b_mipi720x1280_nand_ubi_ubifs_thunderboot_defconfig  27_atk_dlrk3506b_mipi800x1280_nand_ubi_ubifs_defconfig  47_atk_dlrk3506j_autonip1_nand_ubi_ubifs_defconfig
08_atk_dlrk3506b_mipi720x1280_sd_defconfig  28_atk_dlrk3506b_mipi800x1280_sd_defconfig  48_atk_dlrk3506j_rpi024x600_emmc_defconfig
09_atk_dlrk3506j_mipi720x1280_emmc_amlinux_defconfig  29_atk_dlrk3506j_mipi800x1280_emmc_amlinux_defconfig  49_atk_dlrk3506j_rpi024x600_nand_ubi_ubifs_defconfig
10_atk_dlrk3506j_mipi720x1280_emmc_amlinux_defconfig  30_atk_dlrk3506j_mipi800x1280_emmc_amlinux_defconfig  50_atk_dlrk3506j_rpi024x600_nand_ubi_ubifs_defconfig
11_atk_dlrk3506j_mipi720x1280_emmc_defconfig  31_atk_dlrk3506j_mipi800x1280_emmc_defconfig  51_atk_dlrk3506j_rpi024x600_nand_ubi_ubifs_defconfig
12_atk_dlrk3506j_mipi720x1280_emmc_thunderboot_defconfig  32_atk_dlrk3506j_mipi800x1280_emmc_thunderboot_defconfig  52_atk_dlrk3506j_rpi024x600_nand_ubi_ubifs_defconfig
13_atk_dlrk3506j_mipi720x1280_nand_ubi_squashfs_defconfig  33_atk_dlrk3506j_mipi800x1280_nand_ubi_squashfs_defconfig  53_atk_dlrk3506j_rpi024x600_nand_ubi_ubifs_defconfig
14_atk_dlrk3506j_mipi720x1280_nand_ubi_squashfs_thunderboot_defconfig  34_atk_dlrk3506j_mipi800x1280_nand_ubi_squashfs_thunderboot_defconfig  54_atk_dlrk3506j_rpi024x600_nand_ubi_ubifs_defconfig
15_atk_dlrk3506j_mipi720x1280_nand_ubi_ubifs_amlinux_defconfig  35_atk_dlrk3506j_mipi800x1280_nand_ubi_ubifs_amlinux_defconfig  55_atk_dlrk3506j_rpi024x600_nand_ubi_ubifs_defconfig
16_atk_dlrk3506j_mipi720x1280_nand_ubi_ubifs_amlinux_defconfig  36_atk_dlrk3506j_mipi800x1280_nand_ubi_ubifs_amlinux_defconfig  56_atk_dlrk3506j_rpi024x600_nand_ubi_ubifs_defconfig
17_atk_dlrk3506j_mipi720x1280_nand_ubi_ubifs_defconfig  37_atk_dlrk3506j_mipi800x1280_nand_ubi_ubifs_defconfig  57_atk_dlrk3506j_rpi024x600_nand_ubi_ubifs_defconfig
18_atk_dlrk3506j_mipi720x1280_nand_ubi_ubifs_thunderboot_defconfig  38_atk_dlrk3506j_mipi800x1280_nand_ubi_ubifs_thunderboot_defconfig  58_atk_dlrk3506j_rpi024x600_nand_ubi_ubifs_defconfig
19_atk_dlrk3506b_autonip1_nand_ubi_squashfs_defconfig  39_atk_dlrk3506j_rpi024x600_emmc_defconfig  59_atk_dlrk3506j_rpi024x600_nand_ubi_ubifs_defconfig
20_atk_dlrk3506b_autonip1_sd_defconfig  40_atk_dlrk3506j_rpi024x600_nand_ubi_ubifs_defconfig  60_atk_dlrk3506j_rpi024x600_nand_ubi_ubifs_defconfig
alientek@alientek:~/atk-dlrk3506_linux64_release-v1.3.0_20200304/device/rockchip/rk3506$
```

3.1.2 板级配置文件详解

以 `device/rockchip/rk3506/01_atk_dlrk3506b_mipi720x1280_nand_ubi_squashfs_defconfig` 板级配置文件进行解释。由于过长，图未截完整。每个板级文件都有详细注释了，笔者这里就不解释了！

```
# 启用UBI(Unsorted Block Images)文件系统，用于NAND Flash
RK_ROOTFS_UBI=y
# 启用内核模块安装到根文件系统
RK_ROOTFS_INSTALL_MODULES=y
# 注释掉的WiFi/蓝牙芯片型号配置
# RK_WIFIBT_CHIP="AP6256"
# 禁用根文件系统日志守护功能
# RK_ROOTFS_LOG_GUARDIAN is not set
# 禁用WiFi/蓝牙功能
# RK_WIFIBT is not set
# 指定U-Boot配置文件: <SDK>/u-boot/configs/alientek_rk3506_defconfig
# 该配置文件用于正点原子RK3506开发板的U-Boot构建
RK_UBOOT_CFG="alientek_rk3506"
# 启用U-Boot SPL(第二阶段程序加载器)
RK_UBOOT_SPL=y
# 指定内核为ARM32架构
RK_KERNEL_ARM32=y
# 指定内核配置文件: <SDK>/kernel/arch/arm/configs/alientek_rk3506_defconfig
# 该配置文件用于正点原子RK3506开发板的内核构建
RK_KERNEL_CFG="alientek_rk3506_defconfig"
# 内核配置片段文件: <SDK>/kernel/arch/arm/configs/ 目录下的独立配置文件
# 配置片段: 显示、以太网、正点原子板级支持
RK_KERNEL_CFG_FRAGMENT="rk3506-display.config rk3506-ethernet.config rk3506-alientek.config"
# 指定内核设备树文件: <SDK>/kernel/arch/arm/boot/dts/rk3506b-alientek-mipi720x1280-nand-ubi-squashfs.dts
# 该设备树用于RK3506正点原子开发板，配置了720x1280分辨率的MIPI屏幕，NAND Flash存储，支持UBI和SquashFS文件系统
RK_KERNEL_DTS_NAME="rk3506b-alientek-mipi720x1280-nand-ubi-squashfs"
# 启用启动镜像压缩
RK_BOOT_COMPRESSED=y
# 指定压缩启动镜像的ITS文件名
RK_BOOT_FIT_ITS_NAME="zboot.its"
# 指定恢复系统的ITS文件名
RK_RECOVERY_FIT_ITS_NAME="thunderboot4recovery.its"
```

看到上面的配置，太多了，一时间不知道如何下手。没事，我们一个个了解。其中红色的部分是我们开发 u-boot、kernel 和 rootfs 的重要文件。一般情况下用户可以在这些配置文件里新增或者删除一些项来定制自己的系统，其他的配置我们可以酌情了解即可。

```
# 启用 UBI(Unsorted Block Images)文件系统，用于 NAND Flash
RK_ROOTFS_UBI=y
# 启用内核模块安装到根文件系统
RK_ROOTFS_INSTALL_MODULES=y
# 注释掉的 WiFi/蓝牙芯片型号配置
# RK_WIFIBT_CHIP="AP6256"
# 禁用根文件系统日志守护功能
# RK_ROOTFS_LOG_GUARDIAN is not set
# 禁用 WiFi/蓝牙功能
```

```
# RK_WIFIBT is not set
# 指定 U-Boot 配置文件: <SDK>/u-boot/configs/alientek_rk3506_defconfig
# 该配置文件用于正点原子 RK3506 开发板的 U-Boot 构建
RK_UBOOT_CFG="alientek_rk3506"
# 启用 U-Boot SPL(第二阶段程序加载器)
RK_UBOOT_SPL=y
# 指定内核为 ARM32 架构
RK_KERNEL_ARM32=y
# 指定内核配置文件: <SDK>/kernel/arch/arm/configs/alientek_rk3506_defconfig
# 该配置文件用于正点原子 RK3506 开发板的内核构建
RK_KERNEL_CFG="alientek_rk3506_defconfig"
# 内核配置片段文件: <SDK>/kernel/arch/arm/configs/ 目录下的独立配置文件
# 配置片段: 显示、以太网、正点原子板级支持
RK_KERNEL_CFG_FRAGMENTS="rk3506-display.config rk3506-ethernet.config rk3506-
alientek.config"
# 指定内核设备树文件: <SDK>/kernel/arch/arm/boot/dts/rk3506b-alientek-mipi720x1280-
nand-ubi-squashfs.dts
# 该设备树用于 RK3506B 正点原子开发板, 配置了 720x1280 分辨率的 MIPI 屏幕、NAND
Flash 存储, 支持 UBI 和 SquashFS 文件系统
RK_KERNEL_DTS_NAME="rk3506b-alientek-mipi720x1280-nand-ubi-squashfs"
# 启用启动镜像压缩
RK_BOOT_COMPRESSED=y
# 指定压缩启动镜像的 ITS 文件名
RK_BOOT_FIT_ITS_NAME="zboot.its"
# 指定恢复系统的 ITS 文件名
RK_RECOVERY_FIT_ITS_NAME="thunderboot4recovery.its"
# 指定 Flash 总大小为 2GB(2048MB)
RK_FLASH_SIZE=2048
# 第一个额外分区的文件系统类型为 UBI(用于 NAND Flash)
RK_EXTRA_PARTITION_1_FSTYPE="ubi"
# 第一个额外分区的源目录为 rk3506_oem , 对应 :
<SDK>/device/rockchip/rk3506/rk3506_oem/
RK_EXTRA_PARTITION_1_SRC="rk3506_oem"
# 第二个额外分区的文件系统类型为 UBI(用于 NAND Flash)
RK_EXTRA_PARTITION_2_FSTYPE="ubi"
# 指定分区参数文件: <SDK>/device/rockchip/rk3506/parameter-nand.txt
# 该参数文件用于 NAND Flash 的分区配置
RK_PARAMETER="parameter-nand.txt"
# 启用 FIT(扁平化镜像树)镜像格式
RK_USE_FIT_IMG=y
# 指定 Buildroot 基础配置文件: <SDK>/buildroot/configs/alientek_rk3506_defconfig
RK_BUILDROOT_BASE_CFG="alientek_rk3506"
# 启用恢复系统功能
```

```
RK_RECOVERY=y
# 指定恢复系统基础配置文件: <SDK>/buildroot/configs/alientek_rk3506_recovery_defconfig
RK_RECOVERY_BASE_CFG="alientek_rk3506"
# 禁用网络连接检查功能
RK_NETWORK_CHECK=n
# 启用自定义根文件系统主机名
RK_ROOTFS_HOSTNAME_CUSTOM=y
# 指定根文件系统主机名为 ATK-DLRK3506
RK_ROOTFS_HOSTNAME="ATK-DLRK3506"
```

3.2 U-boot 开发

必须执行 2.3 小节选择板级编译文件再编译 Uboot。已经执行过了一次，不用重复执行。清除过 SDK 除外！编译内核及文件系统也一样。

单独编译 u-boot，在 SDK 源码下执行如下指令。

```
./build.sh uboot
```

编译完成如下。

```
Architecture: ARM
Load Address: 0x00001000
Hash algo: sha256
Hash value: 93603ca22cdf22e47ac130e4ac386cdf9474443ab876039807dfc2d5d30b7ecd
Image 2 (fdt)
Description: U-Boot dtb
Created: Wed Mar 4 20:24:15 2026
Type: Flat Device Tree
Compression: uncompressed
Data Size: 6381 Bytes = 6.23 KiB = 0.01 MiB
Architecture: ARM
Hash algo: sha256
Hash value: 178d2c69439c1b947be4ff1825ead1e20d46ba138fabf11f82137bd36f6b2299
Default Configuration: 'conf'
Configuration 0 (conf)
Description: alientek-rk3506
Kernel: unavailable
Firmware: optee
FDT: fdt
Loadables: uboot
*****boot merger ver 1.35*****
Info:Pack loader ok.
creating new idblock from loader...
idblock binary saving at rk3506_idblock_v1.06.111.img
pack loader(SPL) okay! Input: /home/alientek/atk-dlrk3506_linux6.1_release_v1.3.0_20260304/rkbin/RKBOOT/RK3506BMINIALL.tnt
/home/alientek/atk-dlrk3506_linux6.1_release_v1.3.0_20260304/u-boot
pack loader with new: spl/u-boot-spl.bin

Image(no-signed, version=0): uboot.img (FIT with uboot, trust...) is ready
Image(no-signed): rk3506_idblock_v1.06.111.img (with spl, ddr...) is ready
pack uboot.img okay! Input: /home/alientek/atk-dlrk3506_linux6.1_release_v1.3.0_20260304/rkbin/RKTRUST/RK3506TOS.tnt

Platform RK3506 is build OK, with new .config(make alientek_rk3506_defconfig -j24)
/home/alientek/atk-dlrk3506_linux6.1_release_v1.3.0_20260304/prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi/bin/arm-none-linux-gnueabi
Wed Mar 4 20:24:16 CST 2026
+ cd ..
Running 70-loader.sh - build_uboot succeeded.
alientek@alientek:~/atk-dlrk3506_linux6.1_release_v1.3.0_20260304$
```

根据 2.3 小节选择的配置文件，我们开发的时候修改对应的 uboot 设备树和 uboot 的 defconfig 文件即可。注意 uboot 设备树一般无需修改，uboot 启动时加载内核设备树（快速启动模式用的是 SPL + 内核。SPL（Secondary Program Loader）是 U-Boot 源码编译出的第一阶段引导程序）。

3.2.1 如何修改 defconfig

在 3.1.2 小节中 Uboot 我们已经知道正点原子 RK3506 的 Uboot 配置文件为 SDK/u-boot/configs/alientek_rk3506_defconfig。

一种方法是编辑这个 SDK/u-boot/configs/alientek_rk3506_defconfig！

或者有些用户喜欢用 menuconfig 来配置，执行下面的操作！

进入 SDK 源码 u-boot 路径下

```
cd SDK/u-boot/
```

以 alientek_rk3506_defconfig 配置 menuconfig

```
make alientek_rk3506_defconfig ARCH=arm
```

打开 menuconfig, 修改

make menuconfig

最后保存 menuconfig 修改的内容到 configs/rk3506_defconfig。

scripts/kconfig/conf --savedefconfig=configs/alientek_rk3506_defconfig Kconfig

提示以下 warning 不需要理会。

Kconfig: B: warning: environment variable UBOOTVERSION undefined

保存 menuconfig 修改的内容也可以用以下指令。

make savedefconfig

mv defconfig configs/alientek_rk3506_defconfig

若要恢复到出厂配置, 请使用 git 指令恢复! 内核、Buildroot 等其他文件也同理。

git checkout configs/alientek_rk3506_defconfig

3.3 Kernel 开发

对于用户来说, 开发内核常常要修改设备树及 defconfig。这里我们说明用户需要修改哪些设备树及 defconfig。注意, 驱动开发相关请看驱动开发文档。

单独编译内核。

./build.sh kernel

3.3.1 如何修改 defconfig

根据 2.3 小节选择的配置文件和 3.1.2 小节板级配置文件详解, 内核的 defconfig 文件为 **SDK /kernel/arch/arm/configs/alientek_rk3506_defconfig**。

修改方法也非常简单。直接使用 ./build.sh 打开 menuconfig, 修改完成后自动保存到 **SDK /kernel/arch/arm/configs/alientek_rk3506_defconfig** 里。除了 uboot 没有使用 ./build.sh 配置 defconfig, 其他都可以用 build.sh 来保存配置, 建议并且强烈建议使用 build.sh 来修改 defconfig, 保持修改一致性。

./build.sh kernel-config

./build.sh kconfig #或者这个

3.3.2 修改 dts

根据 2.3 小节选择的配置文件和 3.1.2 小节板级配置文件详解, 对于 dts, 用户必须根据选择的板级配置文件使用的 dts 进行修改, 不同的板级配置文件使用的 dts 可能不一样。

atk-dlrk3506b 开发板用到的设备树如下。

ls arch/arm/boot/dts/rk3506b*alientek*.dts

```
alientek@alientek:~/atk-dlrk3506_linux6.1_release_v1.3.0_20260304/kernel$ ls arch/arm/boot/dts/rk3506b*alientek*.dts
arch/arm/boot/dts/rk3506b-alientek-mipi720x1280-nand-ubi-squashfs.dts  arch/arm/boot/dts/rk3506b-alientek-mipi800x1280-nand-ubi-ubifs-amp-linux.dts
arch/arm/boot/dts/rk3506b-alientek-mipi720x1280-nand-ubi-ubifs-amp-linux.dts  arch/arm/boot/dts/rk3506b-alientek-mipi800x1280-nand-ubi-ubifs-amp-mcu.dts
arch/arm/boot/dts/rk3506b-alientek-mipi720x1280-nand-ubi-ubifs-amp-mcu.dts  arch/arm/boot/dts/rk3506b-alientek-mipi800x1280-nand-ubi-ubifs.dts
arch/arm/boot/dts/rk3506b-alientek-mipi720x1280-nand-ubi-ubifs.dts  arch/arm/boot/dts/rk3506b-alientek-mipi800x1280-sd.dts
arch/arm/boot/dts/rk3506b-alientek-mipi720x1280-sd.dts  arch/arm/boot/dts/rk3506b-alientek-rgb1024x600-nand-ubi-ubifs.dts
arch/arm/boot/dts/rk3506b-alientek-mipi800x1280-nand-ubi-squashfs.dts
```

atk-dlrk3506j 开发板用到的设备树如下。

ls arch/arm/boot/dts/rk3506j*alientek*.dts

```
alientek@alientek:~/atk-dlrk3506_linux6.1_release_v1.3.0_20260304/kernel$ ls arch/arm/boot/dts/rk3506j*alientek*.dts
arch/arm/boot/dts/rk3506j-alientek-mipi720x1280-emmc-amp-linux.dts  arch/arm/boot/dts/rk3506j-alientek-mipi800x1280-emmc-amp-mcu.dts
arch/arm/boot/dts/rk3506j-alientek-mipi720x1280-emmc-amp-mcu.dts  arch/arm/boot/dts/rk3506j-alientek-mipi800x1280-emmc.dts
arch/arm/boot/dts/rk3506j-alientek-mipi720x1280-emmc.dts  arch/arm/boot/dts/rk3506j-alientek-mipi800x1280-nand-ubi-squashfs.dts
arch/arm/boot/dts/rk3506j-alientek-mipi720x1280-nand-ubi-squashfs.dts  arch/arm/boot/dts/rk3506j-alientek-mipi800x1280-nand-ubi-ubifs-amp-linux.dts
arch/arm/boot/dts/rk3506j-alientek-mipi720x1280-nand-ubi-ubifs-amp-linux.dts  arch/arm/boot/dts/rk3506j-alientek-mipi800x1280-nand-ubi-ubifs-amp-mcu.dts
arch/arm/boot/dts/rk3506j-alientek-mipi720x1280-nand-ubi-ubifs-amp-mcu.dts  arch/arm/boot/dts/rk3506j-alientek-mipi800x1280-nand-ubi-ubifs.dts
arch/arm/boot/dts/rk3506j-alientek-mipi720x1280-nand-ubi-ubifs.dts  arch/arm/boot/dts/rk3506j-alientek-mipi800x1280-sd.dts
arch/arm/boot/dts/rk3506j-alientek-mipi800x1280-emmc-amp-linux.dts  arch/arm/boot/dts/rk3506j-alientek-rgb1024x600-emmc.dts
arch/arm/boot/dts/rk3506j-alientek-mipi800x1280-emmc.dts  arch/arm/boot/dts/rk3506j-alientek-rgb1024x600-nand-ubi-ubifs.dts
```

设备树解释:

- rk3506b: 瑞芯微 RK3506B 主控
- rk3506j: 瑞芯微 RK3506J 主控
- alientek: 正点原子品牌英文名
- 含 nand 的就是从 nand 存储设备启动系统用的设备树
- 含 “sd/emmc” 代表从 sd/emmc 卡启动系统用的设备树, sd/emmc 卡启动都为可读写文件系统。
- 含 ubi_ubifs 的代表 nand 可读写文件系统。(调试使用)
- 含 ubi_squashfs 代表 nand 只读文件系统。(生产推荐)
- mipi720x1280: 正点原子 5.5 寸 720x1280MIPI 屏幕
- mipi800x1280: 正点原子 10.1 寸 800x1280MIPI 屏幕
- rgb1024x600: 正点原子 7 寸 RGB 屏幕, RK3506B/RK3506J 都需要用户自己做底板, 这里只是配置好了
- amp_linux: 三个 ARM 核心分一个核心跑 RTT, 另外两个 ARM 核心跑 Linux
- amp_mcu: 三个 ARM 核心跑 Linux, MCU (M0) 跑 HAL (BareMetal)

为什么有那么多设备树? 原因之一是屏幕适配, 原因之二是存储适配, 原因之三是读写根文件系统适配, 最后一个原因是 amp 适配, 这样用户无需要手动修改, 降低用户开发难度。用户只需要基于原子板级配置文件和稍微修改一下设备树就可以适配产品了!

3.4 buildroot 开发

对于 buildroot 开发来说, 我们一般配置对应的 defconfig 来定制个人系统, buildroot 详细用法请看网盘资料下的 10、用户手册/02 辅助文档/下的【正点原子】Buildroot 用户手册中文版(正点原子翻译)和【正点原子】Buildroot 快速使用手册文档。

单独编译 Buildroot。

```
./build.sh rootfs
```

3.4.1 如何修改 defconfig

根据 2.3 小节选择的配置文件和 3.1.2 小节板级配置文件详解, 我们知道配置 buildroot 的 defconfig 文件是 SDK/buildroot/configs/alientek_rk3506_defconfig。

这里使用 build.sh 来修改 defconfig。

```
./build.sh buildroot-config
```

```
./build.sh bconfig #或者这个
```

3.4.2 如何重新编译某个 Package

比如用户修改了 qt5base 包的一些配置, 修改了补丁之类的。因为 qt5base 在我们构建 SDK 时已经编译了, Buildroot 会记录这个包已经编译过了, 但是它是不知道你修改了这个包。所以我们需要清除 qt5base 再重新构建 qt5base, 这样 qt5base 的库才会更新! 注意包名可以看 package 下的 (package) 包名或者 dl 下的包名, (package) 包名不带版本号。

```
./build.sh buildroot-make:qt5base-dirclean
```

```
./build.sh rootfs 或者 ./build.sh buildroot-make:qt5base
```

3.4.3 如何添加用户固件

对于用户来说, 正点原子的 SDK 已经功能完善了, 用户只想添加自己的固件到系统中, 那么如何添加呢? 这里介绍两种方法。

方法一:

对于存储用放用户 APP 来说, 一般我们将用户放置系统的/userdata 分区。所以用户可以将固件提前放到 userdata 镜像里, 这样烧录的时候就有用户 App 了。路径为 SDK/device/rockchip/common/extra-parts/userdata/normal。正点原子的 ui 固件也是放到此路径下, 用户可以完全删除 normal 下的所有东西。将 App 放到这个文件夹下, 再次执行 `build.sh` 就会生成 SDK/rockdev/userdata.img。如何启动用户个人 App 相关文档请参考网盘资料下的 10、用户手册/02 辅助文档/下的基于 Buildroot 系统创建自启动程序参考手册。实际上 userdata 是存储数据的, 一般固件不放在这里, 只是 rootfs 空间有限, 对于 256MB 的 NAND 来说, 笔者要为 Userdata 分区留下一部分空间。

方法二:

添加固件到 rootfs 分区下。将用户 App 放到 SDK/buildroot/board/alientek/atk-dlrk3506/fs-overlay 中即可。这个 fs-overlay 就相当于根路径了, 编译 buildroot 时会从此拷贝固件到文件系统中, 请不要删除这里的文件, 防止系统功能不完整! 再次执行 `build.sh` 就会生成 SDK/rockdev/rootfs.img。最后烧写 update.img 即可!

3.5 如何烧写固件

固件指的是整一套系统, 由 Uboot、kernel 和 rootfs 和其他分区镜像文件打包生成的 update.img。

请参考网盘资料下的 10、用户手册/02 辅助文档/下的固件烧写参考手册。将 update.img 烧写到存储设备即可。